

# Aggregation Framework

## Aggregation framework



What is the average student score in each course for 2020?

```
> db.courseResults.aggregate([
  { $match: { "year": 2020 } },
  { $group: { "_id": "$courseId", "avgScore": {
    $avg: "$score" } } }
])
```

- An aggregation framework, which is also sometimes referred to as an aggregation pipeline, is a series of operations that you apply on your data to obtain a sought after outcome. For example, to understand whether students are really developing their knowledge, you want to see the average student scores in 2020 organized by courseid. To obtain this information, you will need to filter the 2020 documents, then group those documents by course, then calculate the average score.

## Course results

```
courseResults = [  
  { _id: 1, courseId: 1, year: 2019, score: 95 },  
  { _id: 2, courseId: 1, year: 2019, score: 84 },  
  { _id: 3, courseId: 2, year: 2019, score: 91 },  
  { _id: 4, courseId: 1, year: 2020, score: 95 },  
  { _id: 5, courseId: 1, year: 2020, score: 100 },  
  { _id: 6, courseId: 2, year: 2020, score: 96 },  
  { _id: 7, courseId: 2, year: 2020, score: 89 },  
  { _id: 8, courseId: 2, year: 2020, score: 73 },  
  { _id: 9, courseId: 2, year: 2021, score: 92 },  
  { _id: 10, courseId: 1, year: 2021, score: 77 },  
]
```

- Let's look at the following sample for courseResults. Each document contains the courseId, the course year, and the score. For simplicity, you will see a subset of the data stored in the document.

## Aggregation stages

```
> db.courseResults.aggregate([  
  { $match: { "year": 2020 } },  
  { $group: { "_id": "$courseId", "avgScore": { $avg: "$score" } } }  
)
```

- This example uses the following aggregation stages, \$match to filter out documents from year 2020. And \$group to group documents from the last

stage using `courseId`, and to calculate the average score in the field name `average score`.

## Course results

---

```
courseResults = [  
  { _id: 1, courseId: 1, year: 2019, score: 95 },  
  { _id: 2, courseId: 1, year: 2019, score: 84 },  
  { _id: 3, courseId: 2, year: 2019, score: 91 },  
  { _id: 4, courseId: 1, year: 2020, score: 95 },  
  { _id: 5, courseId: 1, year: 2020, score: 100 },  
  { _id: 6, courseId: 2, year: 2020, score: 96 },  
  { _id: 7, courseId: 2, year: 2020, score: 89 },  
  { _id: 8, courseId: 2, year: 2020, score: 73 },  
  { _id: 9, courseId: 2, year: 2021, score: 92 },  
  { _id: 10, courseId: 1, year: 2021, score: 77 },  
]
```

- Processing this code is like running a pipeline. Documents enter from one side, go through one or more processes, and come out in the sought after format at the other end.

## Aggregation stages

```
> db.courseResults.aggregate([
  { $match: { "year": 2020 } },
  { $group: { "_id": "$courseId", "avgScore": { $avg: "$score" } } }
])
```

- Let's review the following sample data. When you apply the \$match stage, MongoDB filters out those documents that do not have the year 2020. MongoDB only takes documents that meet all criteria to the next stage. Next the \$group stage separates documents into groups according to a group key. A group key is often a field or group of fields, you'll use the id field to set the group key.
- The output is one document for each unique group key. In this example, the group key is the courseid. The dollar sign means value from the document. Average score is a field that will hold the average score. Dollar average returns the average of the numeric values in the field score, represented as dollar scores. The output will display two documents, one document for courseid 1, and one document for courseid 2. You'll also see each course's average course scores for the year 2020.

## Common aggregation stages

---



- \$project
- \$sort
- \$count
- \$merge

## Applying \$project

---

```
students = [  
  { "_id": 1, "firstName": "Sophie", "lastName": "Foucher", "city": "Quebo"  
  },  
  { "_id": 2, "firstName": "Stafford", "lastName": "von Hagt", "city":  
    "Terezin" },  
  { "_id": 3, "firstName": "Olivero", "lastName": "Liggett", "city":  
    "Guaranda" }  
]
```

## Applying \$project: Resulting outcomes

```
[
  { "_id": 1, "firstName": "Sophie", "lastName": "Foucher" },
  { "_id": 2, "firstName": "Stafford", "lastName": "von Hagt" },
  { "_id": 3, "firstName": "Olivero", "lastName": "Liggett" }
]
```

## Applying the \$sort stage: Sorting order options

\$sort

```
db.students.aggregate([
  {
    $project: { firstName: 1, lastName: 1 }
  },
  {
    $sort: { lastName: -1 }
  }
])
```

## Applying \$sort stage: The resulting output

```
[{
  "_id": 2,
  "firstName": "Stafford", "lastName": "von Hagt"
},
{
  "_id": 3,
  "firstName": "Olivero", "lastName": "Liggett"
},
{
  "_id": 1,
  "firstName": "Sophie", "lastName": "Foucher"
}]
```

- An aggregation stage can be repeated in a pipeline, and the most common stages are \$project stage reshapes documents by including or excluding fields. \$sort stage sorts the documents in requested order by field. \$counts documents in that stage, and \$merge to move output to a collection.
- Let's use this student's data to see dollar project in action.
- You apply \$project stage to only take first and lastName when using dollar project. With First and lastName, you only see those fields and id is always included unless explicitly excluded. You can add \$sort to always see the results in a specific order.
- You can specify a sort order based on any field in the document at that stage of pipeline. For this example, you are sorting by last name in descending order, which is represented by -1.
- For ascending order, you use a 1 without a negative mark. The output displays student documents with first and last name along with underscore id with the students in descending order on the last name.

## Applying the \$count stage

\$count

```
> db.students.aggregate([{$count: "totalStudents"}])
{ "totalStudents": 3 }
```

- Use \$count to calculate the total number of students and assign the outcome to a field called totalStudents. Which produces the outcome of totalStudents as 3.

## Applying \$match

```
courseResults = [
  { _id: 1, courseId: 1, year: 2019, score: 95 },
  { _id: 2, courseId: 1, year: 2019, score: 84 },
  { _id: 3, courseId: 2, year: 2019, score: 91 },
  { _id: 4, courseId: 1, year: 2020, score: 95 },
  { _id: 5, courseId: 1, year: 2020, score: 100 },
  { _id: 6, courseId: 2, year: 2020, score: 96 },
  { _id: 7, courseId: 2, year: 2020, score: 89 },
  { _id: 8, courseId: 2, year: 2020, score: 73 },
  { _id: 9, courseId: 2, year: 2021, score: 92 },
  { _id: 10, courseId: 1, year: 2021, score: 77 }]
```

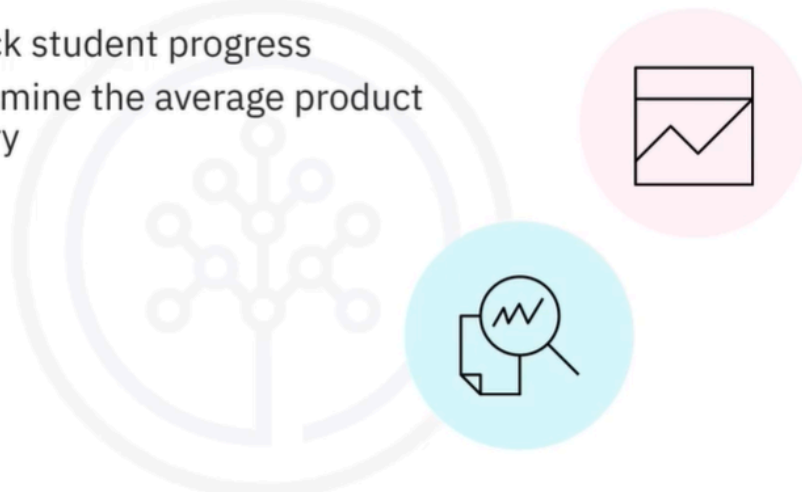
- Returning to the course results example, let's store the output into a collection called average scores. You accomplish this task using \$merge. You won't see

the output from \$merge stage in your client because the output documents are stored in the target collection.

## Aggregation use cases

---

- Reporting: Track student progress
- Analysis: Determine the average product sales by country



- Now that you are familiar with different stages that are available in the MongoDB aggregation pipeline, let's consider when you can use an aggregation framework. As you saw earlier, you can use an aggregation framework to track the progress of students by course.
- You could also apply an aggregation framework to an ecommerce application to calculate the average sale of a product per country. You can create simplified calculations, complex calculations, complex groupings, and simple groupings of data to provide multiple perspectives about the data.

## What is the purpose of the aggregation framework in MongoDB?

The purpose of the aggregation framework in MongoDB is to **process and analyze data** in a structured way. Here are the key points:

- **Data Transformation:** It allows you to transform data into a desired format through a series of operations.
- **Complex Queries:** You can perform complex queries that involve filtering, grouping, and calculating values, which is useful for data analysis.
- **Performance:** It is optimized for performance, enabling efficient processing of large datasets.
- **Insights:** Helps in deriving insights from data, such as averages, totals, and other statistical measures.

In summary, the aggregation framework is essential for performing detailed data analysis and obtaining meaningful results from your MongoDB collections.

## What is the most difficult concept in aggregation framework?

One of the more challenging concepts in the aggregation framework can be the **\$group stage**. Here's why:

- **Grouping Logic:** Understanding how to define the group key (the field by which you want to group your data) can be tricky. You need to know which field will provide meaningful insights.
- **Aggregation Operators:** Using aggregation operators like `$avg`, `$sum`, `$max`, and `$min` correctly requires a good grasp of what each operator does and how to apply them to your grouped data.
- **Nested Grouping:** Sometimes, you may need to perform multiple levels of grouping or combine it with other stages, which can complicate the logic and make it harder to follow.

Overall, mastering the \$group stage is crucial for effectively using the aggregation framework, as it is central to summarizing and analyzing data.

## What challenges might arise when using nested grouping in aggregation?

When using nested grouping in the aggregation framework, several challenges can arise:

- **Complexity:** The logic can become complicated, making it harder to understand and maintain. You need to carefully track how data flows through each grouping stage.
- **Performance:** Nested grouping can lead to performance issues, especially with large datasets. Each additional layer of grouping requires more processing power and time.
- **Data Structure:** Ensuring that the data structure supports the desired nested grouping can be challenging. You may need to reshape your data or adjust your queries accordingly.
- **Debugging:** Identifying errors or unexpected results can be difficult. If the output isn't what you expect, tracing back through multiple nested stages to find the issue can be time-consuming.
- **Memory Usage:** More complex aggregations may consume more memory, which can lead to limitations if the dataset is large.

To effectively manage these challenges, it's important to plan your aggregation pipeline carefully and test it with smaller datasets before scaling up.

## How can you simplify nested grouping in your queries?

To simplify nested grouping in your aggregation queries, consider the following strategies:

- **Break Down the Pipeline:** Divide your aggregation pipeline into smaller, manageable stages. This makes it easier to understand each part of the process.

- **Use Intermediate Collections:** Store intermediate results in temporary collections. This allows you to perform complex groupings step by step, rather than all at once.
- **Clear Naming Conventions:** Use clear and descriptive names for your fields and variables. This helps in understanding the purpose of each grouping and makes the query more readable.
- **Comment Your Code:** Add comments to explain the logic behind each stage of your aggregation. This can be helpful for future reference or for others who may read your code.
- **Test Incrementally:** Test each stage of your aggregation pipeline with sample data before combining them. This helps identify issues early and ensures each part works as intended.
- **Limit the Data:** If possible, filter your data before grouping to reduce the dataset size. This can improve performance and make the grouping process simpler.

By applying these strategies, you can make nested grouping in your queries more manageable and easier to work with.